



SÃO
PAULO
TECH
SCHOOL

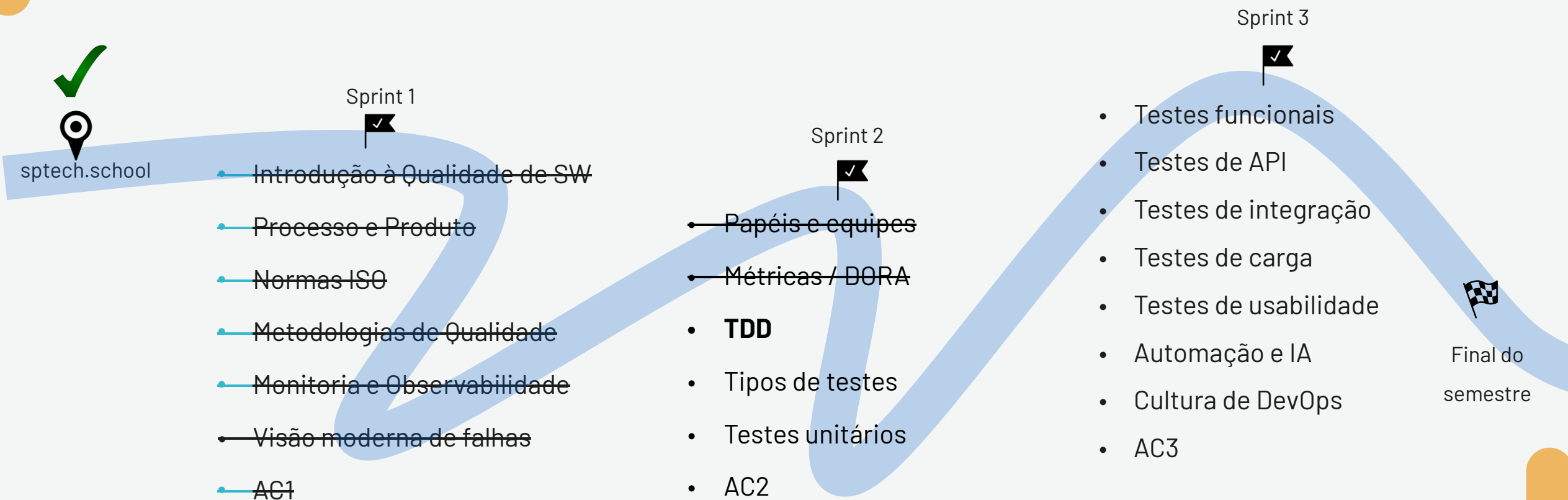
Qualidade de Software

TDD - aula extra

31/mar

Prates

Nosso caminho - Ementa



Tópicos da Aula

TDD - Test Driven Development

Exemplos e exercício de reforço

TDD - Test Driven Development

The background of the slide is a dark, almost black, field filled with intricate, flowing white lines and dots. These elements create a sense of movement and depth, resembling a stylized representation of a network or a celestial map. The lines are thin and curved, weaving across the frame, while the dots are small and scattered, some appearing as bright points of light. The overall effect is a complex, organic pattern that frames the central text.



TDD - Test Driven Development

Conceito

- Escrever os testes **antes** do código de produção
- Processo cíclico: Red → Green → Refactor

Ciclo

- Escreva um teste que falha (Red)
- Escreva o código mínimo para passar o teste (Green)
- Refatore o código mantendo os testes verdes (Refactor)



Exemplo - validação de função Soma

RED - teste falhando

TDD1.PY

```
def teste_soma():  
    assert soma(2, 3) == 5
```


Exemplo - validação de função Soma

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest tdd1.py]
===== test session starts =====
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
collected 1 item

tdd1.py F [100%]

===== FAILURES =====
RED - test_soma

def test_soma():
>     assert soma(2, 3) == 5
E       NameError: name 'soma' is not defined

tdd1.py:2: NameError

===== short test summary info =====
FAILED tdd1.py::test_soma - NameError: name 'soma' is not defined
===== 1 failed in 0.08s =====
```




Exemplo - validação de função Soma

GREEN - teste passando ok

SOMA.PY

```
def soma(a: int, b: int) -> int:  
    return a + b
```



Exemplo - validação de função Soma

GREEN - teste passando ok

TDD1.PY

```
from soma import soma
```

```
def teste_soma():  
    assert soma(2, 3) == 5
```

Exemplo - validação de função Soma

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest tdd1.py]
```

```
===== test session starts =====
```

```
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
```

```
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
```

```
collected 1 item
```

```
tdd1.py .
```

```
[100%]
```

```
===== 1 passed in 0.01s =====
```



Exemplo - validação de função Soma

REFACTOR - adicionando mais testes / revisando

```
from soma import soma
```

```
def test_soma():  
    assert soma(2,3) == 5
```

```
def test_somaNegativo():  
    assert soma(-1, -5) == -6
```

Exemplo - validação de função Soma

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest tdd1.py
```

```
===== test session starts =====
```

```
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
```

```
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
```

```
collected 2 items
```

```
tdd1.py ..
```

```
[100%]
```

```
===== 2 passed in 0.04s =====
```




Erros mais comuns

- Fazer o teste dar erro por algum problema no código, e não pq não tem a implementação
- Fazer vários testes de início, ao invés de 1
- Já escrever a implementação da função que será testada, antes ou junto com o teste
- Confundir o teste com a função que será testada
- Não seguir os passos Red > Green > Refactor
- Fazer o código mínimo para o teste passar não é dar um `return True`

Exemplo - erros mais comuns

- Fazer o teste dar erro por algum problema no código, e não pq não tem a implementação

```
from cpf import valida_cpf
```

```
def test_valida_cpf():  
    assert valida_cpf("12345678901") == true
```

NameError: name true is not defined

Exemplo - erros mais comuns

- Fazer vários testes de início, ao invés de 1

```
def test_valida_cpf():  
    assert valida_cpf("12345678901") == True
```

```
def test_valida_cpf_letras():  
    assert valida_cpf("123ABC78901") == False
```

```
def test_valida_cpf_menos_digitos():  
    assert valida_cpf("123901") == False
```

Exemplo - erros mais comuns

- Já escrever a implementação da função que será testada, antes ou junto com o teste

```
def test_valida_cpf():  
    assert valida_cpf("12345678901") == True
```

```
def valida_cpf(cpf: str) -> bool:  
    return True
```

Exemplo - erros mais comuns

- Confundir o teste com a função que será testada

```
def test_valida_cpf(cpf: str) -> bool:  
    assert valida_cpf("12345678901") == True  
  
    if not cpf.isdigit() or len(cpf) != 11:  
        return False  
  
    if cpf == cpf[0] * 11:  
        return False
```

Exemplo - erros mais comuns

- Não seguir os passos Red > Green > Refactor

```
def test_valida_cpf(cpf: str) -> bool:  
    assert valida_cpf("12345678901") == True
```

```
def valida_cpf(cpf: str) -> bool:  
    if not cpf.isdigit() or len(cpf) != 11:  
        return False  
    if cpf == cpf[0] * 11:  
        return False
```

Exemplo - erros mais comuns

- Fazer o código mínimo para o teste passar não é dar um return True

```
def test_valida_cpf(cpf: str) -> bool:  
    assert valida_cpf("12345678901") == True
```

```
def valida_cpf(cpf: str) -> bool:  
    return True
```

Atividade 1

**Implementar uma
função de validação se
uma operação de CRUD
foi bem sucedida**



Atividade - validação de CRUD

Instruções

- Implementar uma classe Pessoa, com atributos Nome e CPF, gravando ela localmente (qualquer forma de armazenamento: bd, disco ou memória)
- Use o ciclo de TDD para testar:
 - Escreva um teste que falha, para testar se a Pessoa foi criada corretamente
 - Implemente o código mínimo para passar o teste
 - Refatore para adicionar outras validações (se o campo CPF é numérico)
- Gere evidências de cada passo (RED > GREEN > REFACTOR)

Resolução - passo RED

- Escrever um caso de teste para testar a classe Pessoa

```
def test_Pessoa():  
    someone = Pessoa("Nome", "12345678901")  
    assert someone.nome == "Nome"  
    assert someone.cpf == "12345678901"
```

Resolução - passo RED

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest test_crud.py]
===== test session starts =====
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
collected 1 item

test_crud.py F [100%]

===== FAILURES =====
test_Pessoa

    def test_Pessoa():
>     someone = Pessoa("Rodrigo Prates", "27604729819")
E     NameError: name 'Pessoa' is not defined

test_crud.py:3: NameError

===== short test summary info =====
FAILED test_crud.py::test_Pessoa - NameError: name 'Pessoa' is not defined
===== 1 failed in 0.07s =====
```

Resolução - passo GREEN

- Implemente o código mínimo para o teste passar (test_crud.py)

```
from crud import Pessoa
```

```
def test_Pessoa():
```

```
    someone = Pessoa("Nome", "12345678901")
```

```
    assert someone.nome == "Nome"
```

```
    assert someone.cpf == "12345678901"
```

Resolução - passo GREEN

- Implemente o código mínimo para o teste passar (crud.py)

```
class Pessoa():  
    def __init__(self, nome: str, cpf: str):  
        self.nome = nome  
        self.cpf = cpf  
  
    def valida_cpf(self) -> bool:  
        if not self.cpf.isdigit() or len(self.cpf) != 11:  
            return False  
        return True
```

Resolução - passo GREEN

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest test_crud.py]
===== test session starts =====
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
collected 1 item

test_crud.py . [100%]

def test_Pessoa():
    someone = Pessoa("Nome", "12345678901")

===== 1 passed in 0.01s =====
```

Resolução - passo REFACTOR

- Refatore o código para adicionar mais validações e complementar

```
from crud import Pessoa
```

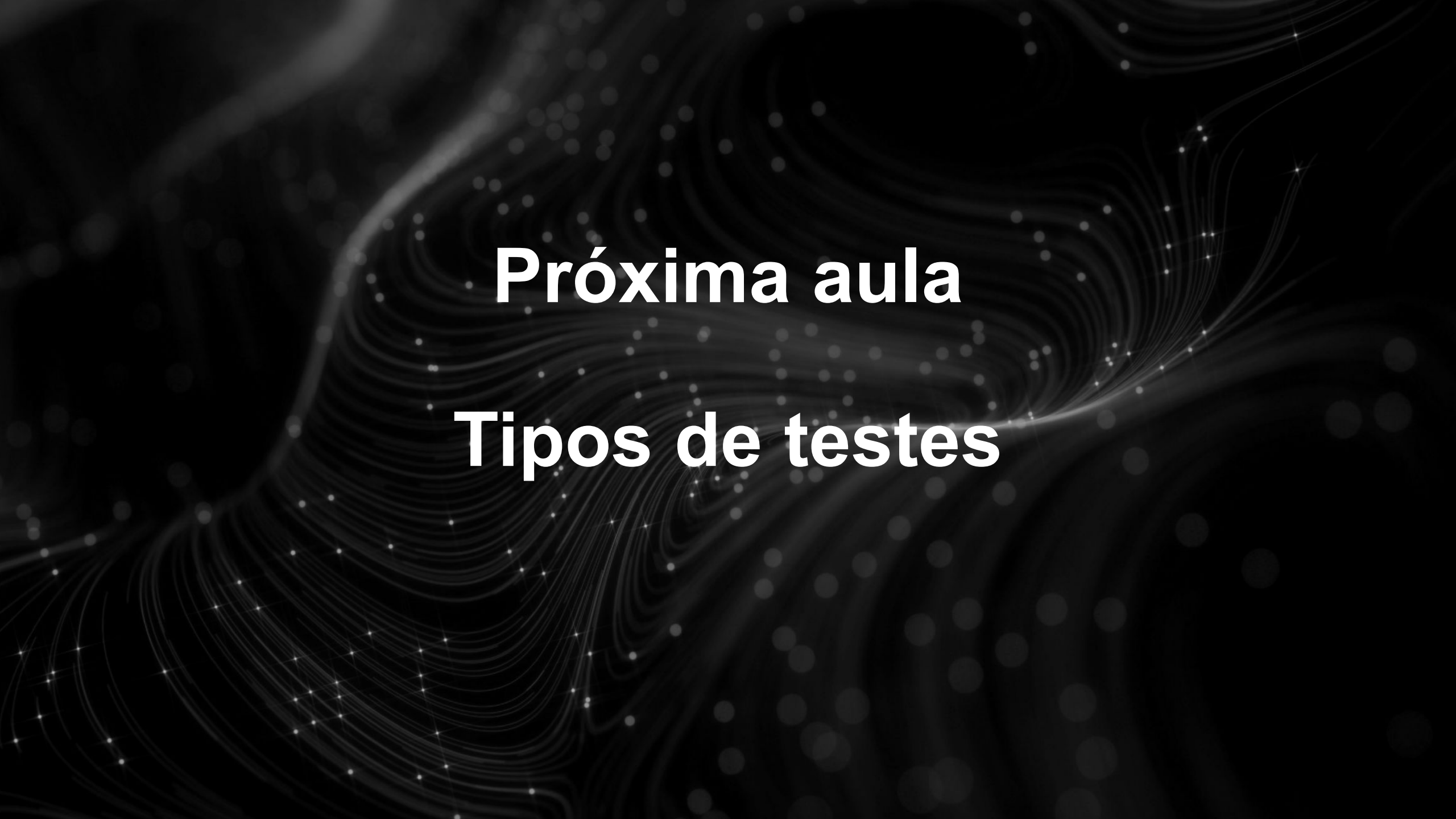
```
def test_Pessoa():
```

```
    someone = Pessoa("Nome", "12345678901")
```

```
    assert someone.nome == "Nome"
```

```
    assert someone.cpf == "12345678901"
```

```
    assert someone.valida_cpf() == True
```


The background is a dark, textured surface with intricate, flowing, and swirling patterns of light gray lines. These lines create a sense of movement and depth, resembling a nebula or a complex data visualization. Scattered throughout the background are numerous small, out-of-focus light spots, known as bokeh, which vary in size and brightness, adding to the ethereal and futuristic feel of the image.

Próxima aula

Tipos de testes



Agradeço a sua atenção!

Qualidade de Software

Professor Prates

Aula 7

SÃO
PAULO
TECH
SCHOOL